Communication Costs and Verification of a Step-Controlled FPGA CGRA

Mattia Fait
University of Trento
Trento, Italy

***Abstract*—Small coarse-grained reconfigurable arrays (CGRAs) attached to a host over a serial link are common in FPGA prototyping, yet their cost is usually reported as fabric resources and clock frequency. We study a 16-bit, step-controlled $R \times C$ CGRA ($RC \leq 16$) driven over UART, and ask three questions: when fusing a write–run–read sequence into one transaction pays for its extra bytes; how geometry and mapping determine traffic and useful work; and how keeping operands resident on chip changes the balance between communication, resources and verification effort. Exact byte, transaction and step counts from the public API, checked against scalar references and closed forms, feed a first-order link model. Fusion reduces transactions by 1.99–3.95× and is never slower for two of five kernels; for the other three it wins only when the per-transaction turnaround exceeds 58–174 μs. Every mapping performs useful work in at most one of R PE-step slots, often one of 16, and on the default 4×4 mesh the modelled device time is below $3 \cdot 10^{-5}$ of the link time. Batched resident operands reduce dot-product traffic 4.6× without reuse and 26.9× for eight identical calls, at +68.6% LUTs and +320% flip-flops. Safety properties are proved on the production VHDL, and the proved capture guards are linked to the multicycle exceptions checked by static timing analysis. All results are pre-silicon; no board measurement was performed.**

***Index Terms*—Coarse-grained reconfigurable array, FPGA, host–accelerator communication, performance modelling, formal verification, multicycle constraints.**

I. Introduction

COARSE-GRAINED reconfigurable arrays (CGRAs) configure word-level operations and nearest-neighbour communication at run time [1]–[3]. Small instances are frequently implemented as FPGA overlays or prototypes and attached to a host processor [4], [5]. Their evaluation commonly reports resources, clock frequency and kernel cycles. For an accelerator reached through a narrow host link, however, the number of bytes and round trips can dominate, and the timing argument that lets a datapath settle over several clock cycles must be justified by the control logic rather than assumed.

This paper is a reproducible pre-silicon study of these two concerns on one concrete design: a 16-bit, step-controlled $R \times C$ mesh ($RC \leq 16$, default 4×4) with one result register per processing element (PE), operands injected at the north and west edges, and a UART command interface. The mesh, the UART, CRC framing, response caching and the C API are conventional building blocks and are not claimed as architectural novelty. We ask:

RQ1 When does fusing a write–run–read sequence into one transaction compensate for its larger requests?

RQ2 How do geometry and mapping determine wire traffic, useful operations and PE occupancy?

RQ3 How does keeping operands resident on chip change the balance between communication, resources and verification effort?

The contributions are an explicit accounting and its evidence rather than a new fabric: (i) exact byte, transaction and array-step counts for five kernels, two protocol versions and five 16-PE geometries, obtained through the public API, checked against independent scalar results and against closed forms derived from the protocol (Sections IV and VI); (ii) a first-order link model that separates serialization, per-transaction turnaround and device time, with break-even turnarounds for fusion; (iii) a decomposition of the benefit of resident operands into batching and reuse, together with its area cost; and (iv) a documented chain from formally proved capture guards on the production VHDL to the multicycle timing exceptions applied by static timing analysis (STA). No result was measured on an FPGA board; Section VII lists the evidence that broader claims would require.

This is a pre-silicon study: no FPGA board was available, and no quantity in this paper was measured on hardware. Section VIII describes the artifact and how each result is regenerated.

II. Related Work

CGRAs trade the bit-level flexibility of FPGAs for word-level operations and fewer configuration bits [1]–[3]. Classic designs differ mainly in how much storage and communication each element has: ADRES couples a VLIW processor to a reconfigurable matrix [6], MorphoSys provides richer communication resources [7], HyCUBE adds reconfigurable multi-hop paths [8], PipeRench virtualises pipelined computations [9], DySER specialises datapaths inside a processor [10], and Plasticine organises compute and memory around parallel patterns [11].

Table I compares the work closest in purpose. CGRA-ME generates RTL from an architecture description and evaluates CGRA variants both as standard-cell designs and as FPGA overlays [4], [12]. QuickDough is a soft CGRA overlay whose PEs hold instruction and data memories; host data reach on-chip buffers by DMA, and accelerator performance is estimated from schedule cycles, implementation frequency and transfer latencies of a Zynq platform [5]. OpenEdgeCGRA is an open-hardware CGRA with several registers per PE, a torus interconnect and per-column program counters, integrated in a microcontroller platform [13]; Aspros et al. estimate energy and

latency of its kernel executions and compare them with post-synthesis simulation [14]. These designs are more capable than ours (local storage, instruction sequencing, memory access); none of them is evaluated here, and resources or frequencies obtained with different devices, precisions, geometries and tool flows are not comparable with ours. What this paper adds is not a better fabric but a complete accounting of a deliberately minimal one: every byte, transaction and step on a narrow host link, and a traceable link between control-logic proofs and timing exceptions.

A. Systolic computation and stationary data

Systolic computation coordinates local operations and communication over successive steps [15], [16]; modern matrix engines such as the TPU use this principle at a much larger scale [17]. A stationary-data label describes which value remains local while others move. In an *output-stationary* mapping, an output's partial sum stays in local storage as its products accumulate. In a *weight-stationary* mapping, weights remain local for reuse [18]. Eyeriss's row-stationary organisation exploits several kinds of reuse [19]. These labels describe data movement; they do not by themselves specify a particular number of registers or an interconnect.

Our MAC instruction already supports local output accumulation. The limiting resource is independent forwarding: a PE exposes only its result register to neighbours. Section IV-A explains why the implemented matrix mapping uses stationary vector coefficients, product delay lines and host reduction. It does not establish that every output-stationary schedule is impossible on the mesh.

III. System Under Study

The fabric is an $R \times C$ mesh of identical PEs on 16-bit two's-complement words; the integrated controller, protocol and host support every positive geometry with $RC \leq 16$, fixed at elaboration (default 4×4). All PEs share one clock. A global *step* enable, issued by the controller, advances every PE register at once; the host requests a number of steps per command, so execution is deterministic and counted in steps rather than in cycles. Consecutive steps are separated by $D = \texttt{G_STEP_DIV}$ clocks (default 2), which Section VI-D uses to justify a D-cycle timing exception.

A. Processing element and mesh

Each PE (Fig. 1) has two 8-way operand multiplexers selecting a neighbour's registered output (north, south, east, west), a 16-bit immediate, its own register or zero; the eighth code is reserved and yields zero. A 4-bit opcode selects one of sixteen operations: no-op, pass, add, subtract, multiply and multiply–accumulate (low 16 bits of the product), accumulate, bitwise AND/OR/XOR, logical left shift and arithmetic right shift by the low four bits of the second operand, signed maximum, minimum and absolute value, and constant load. Arithmetic wraps modulo 2^{16}; in particular $|-32768| = -32768$. A 32-bit configuration word holds the immediate (bits 15–0), opcode (19–16) and selects (22–20, 25–23); bits 31–26 are unused. The single result register is the only state of a PE.

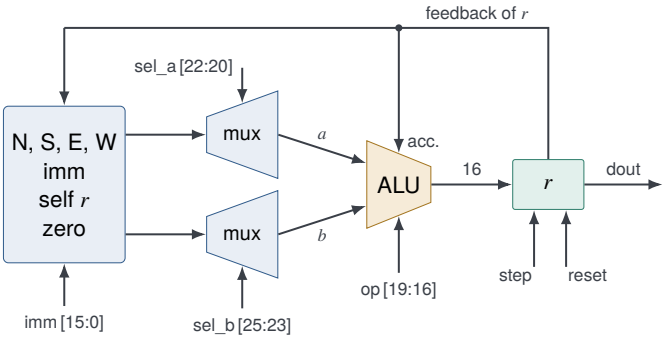


Fig. 1. Processing-element datapath. Two 8-way multiplexers select each operand from the four neighbours, the immediate, the PE's own register or zero; configuration fields are shown with their bit ranges. MAC and ACC also read r directly. The register captures on *step* and is cleared by reset.

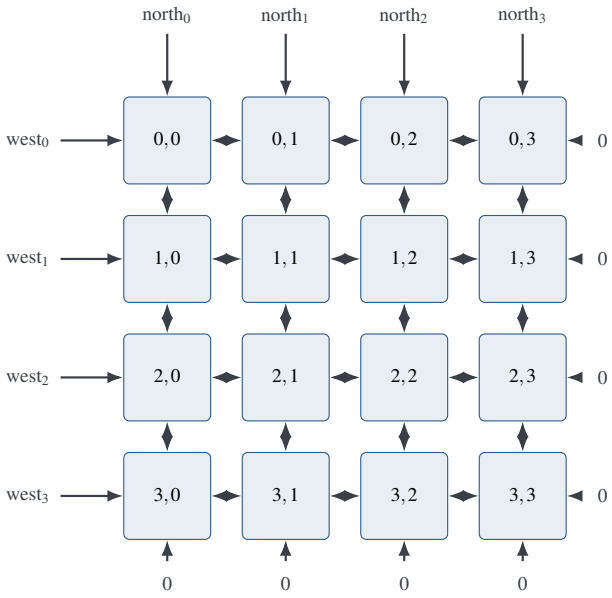


Fig. 2. The 4×4 mesh. Solid arrows are nearest-neighbour register connections; operands are injected only at the north edge (per column) and the west edge (per row), while the south and east edges read zero. This edge-only injection is the central constraint on dataflow mapping (Section IV-A).

Each PE reads the registered outputs of its four neighbours, so a value advances at most one hop per step (Fig. 2). External operands enter only at two edges: north input c feeds $\mathrm{PE}(0,c)$ and west input r feeds $\mathrm{PE}(r,0)$; the south and east borders read zero. All RC registers can be read back. Because a PE exposes one register to all four neighbours, it cannot forward two independent operands in the same step. This constrains schedules—it does not make output-stationary mappings impossible (Section IV-A)—and it is the reason why several mappings below spend PEs on forwarding.

TABLE I

COMPARISON WITH RELATED CGRA WORK, AS DOCUMENTED IN THE CITED PRIMARY SOURCES. "NOT REPORTED" MEANS NOT FOUND IN THOSE SOURCES, NOT ABSENT FROM THE DESIGN. RESOURCES AND FREQUENCIES ARE DELIBERATELY OMITTED: THEY COME FROM DIFFERENT DEVICES, PRECISIONS AND FLOWS.

Work	PE state and contexts	Communication and memory	Host interface	Mapping	Validation reported
CGRA-ME [4], [12]	Described per architecture (XML)	Architecture-defined	Not reported	Integrated mapper	Standard-cell and FPGA-overlay implementation
QuickDough [5]	Instruction memory and multi-port data memory per PE	Torus; on-chip I/O buffers	DMA from an ARM host (Zynq)	Compile-time scheduling of loop DFGs	Analytical performance from schedule, $f_{\max}$ and transfer latency
OpenEdgeCGRA [13]	Four registers plus output register; per-column program counter	Torus; column memory ports	Microcontroller integration (X-HEEP)	Kernel assembly, behavioural simulator	Not reported in the repository; post-synthesis comparison in [14]
This work	One result register; one configuration	Mesh with north/west edge injection; optional two 1024×16 banks	UART, 115 200 Bd	Fixed library mappings with host reduction	Exact counters, RTL replay, formal safety proofs, STA; no board

TABLE II

LEGACY COMMAND SET. `EXEC` IS THE VERSION-3 ADDITION; THE OTHER SIX COMMANDS FORM VERSION 2. "CK": 8-BIT ADDITIVE CHECKSUM.

Cmd	Payload	Reply
`ID`	—	signature, version, geometry
`CFG`	RC words + ck	ACK / NACK
`WR`	$(R+C)$ words + ck	ACK / NACK
`RUN`	step count N	ACK (after N steps)
`RD`	—	RC registers + ck
`RST`	—	ACK (clears registers)
`EXEC`	N, flags, tap mask, $(R+C)$ words + ck	masked registers + ck + status

B. Command protocols

The legacy protocol (Table II) uses one-byte commands whose payload lengths follow from the geometry. Version 2 has six commands; version 3 adds `EXEC`, which carries the step count, a flags byte (bit 0 clears the PE registers first), a 16-bit tap mask and the edge operands, and returns only the selected registers with a checksum and a status byte (Fig. 3). Configuration and read-back use row-major indices $rC+c$; an input payload holds all R west words followed by all C north words. The identification reply reports version and geometry, so one host binary drives either version and any supported geometry.

C. Failure contracts of the legacy protocol

The legacy protocol has no delimiter or sequence number, and its 8-bit additive checksum detects only some corruptions. Its observable contract, checked by differential replay against the RTL, is deliberately weak. *(i)* `CFG` and `WR` commit words as they arrive, before the checksum is validated: a `NACK` reports a failed transfer but does not roll back. A rejected `EXEC` likewise latches its inputs, but neither clears nor steps the array. *(ii)* Clearing the datapath (`RST` or the `EXEC` flag) and restoring the parser to its idle state are different operations: only a device reset or the receive watchdog (10^8 clocks, 1 s at 100 MHz) returns a parser that waits for payload bytes to idle.

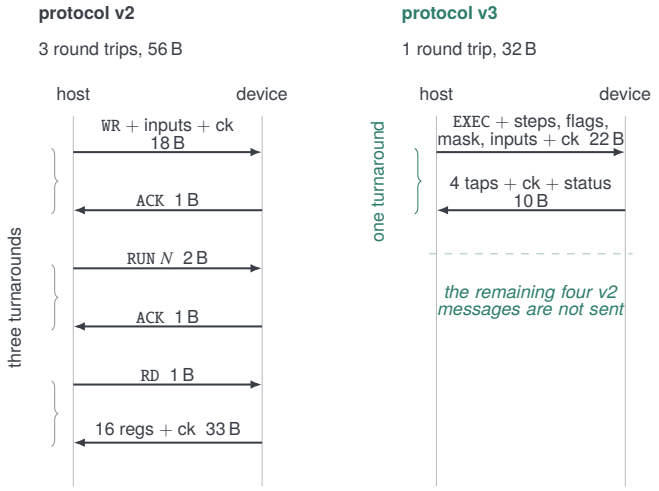


Fig. 3. One element-wise chunk on the wire. The unfused sequence (left) spends three turnarounds and returns all sixteen registers; `EXEC` (right) spends one and returns only the four registers the tap mask names. The optional datapath clear is folded into the flags byte. Reply framing assumes that the command boundary and tap mask arrive intact.

(iii) A checksum error inside an intact frame is recoverable; loss of framing (an inserted, dropped or corrupted length-defining byte) is not detected in general. The library therefore retransmits only after a complete, checksum-valid `NACK` of `CFG`, `WR` or a resetting `EXEC`; a timeout, I/O error or malformed reply poisons the handle, and later calls fail without sending bytes. Reopening the serial port does not reset the device.

D. Experimental resident-block extension

An optional extension adds two 1024×16 operand banks and a framed protocol for one kernel, a resident dot product; it is elaborated only when the generic `G_BLOCK_ENABLE` is set. A single preflight byte (`0x08`), legal only when the legacy parser is idle, selects it; an older controller answers `NACK`, so probing cannot make old RTL interpret framed payload. Framed mode

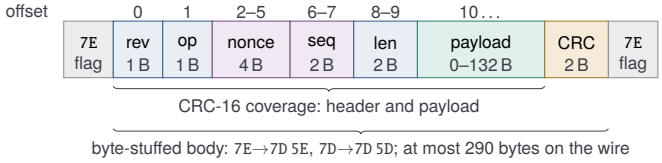


Fig. 4. One resident-block frame. The ten-byte header identifies the session (nonce) and request (sequence); the CRC is computed before stuffing. A reply echoes the opcode with bit 7 set and begins its payload with a status byte. Field widths are schematic.

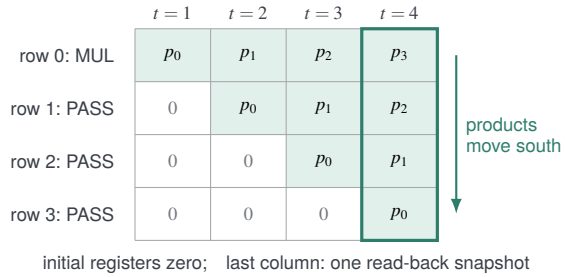


Fig. 5. One column of a 4×4 matrix tile. Here $p_i = A_{i,c}x_c$. Time increases from left to right; the last column is the read-back snapshot. The host sums snapshot PE rows across all columns to obtain output rows in reverse order.

then persists until an external reset; there is no in-band return to legacy decoding.

Each frame (Fig. 4) is delimited by `0x7E` and byte-stuffed as in HDLC-like framing [20]. A ten-byte header carries revision, opcode, a 32-bit session nonce, a 16-bit sequence number and the payload length; a CRC-16 (polynomial `0x1021`, initial value `0xFFFF`, no reflection) covers header and payload [21], [22]. Revision, length, canonical escaping and CRC are validated before a request is acted upon; rejected frames produce neither a reply nor a committed effect. CRC detects errors; it is not authentication [23], and a corrupted frame that happens to validate is indistinguishable from a legitimate one.

The session semantics provide *at-most-once effects with response replay* within narrow conditions [24]. One request may be outstanding, and only the response to the immediately preceding request is cached. A `HELLO` with a fresh nonce starts a session at sequence 1 and invalidates resident data. A request whose opcode, nonce, sequence, length and payload are identical to the cached one receives the cached response without re-execution; the same sequence with different contents receives `CONFLICT`; only the next sequence number executes. Sequence 65 535 exhausts the session; there is no wraparound. An external reset loses the cache, and a random nonce makes accidental reuse unlikely but not impossible. Within one serialised connection between resets, a host that retransmits the exact frame after a lost reply therefore obtains the original response without a duplicated upload or computation. The public API does not retransmit automatically: it poisons the handle after an uncertain transfer, and the replay facility is available only to clients that implement it.

Commands are `UPLOAD` (up to 64 words appended to a bank and committed atomically per frame), `DOT` n and `RESULT`. During `DOT` a sequencer owns the array, configures $\mathrm{PE}(0,0)$ as a MAC on its edge inputs, resets the array once and issues exactly n captures, each $D+3$ clocks apart; synchronous RAM outputs pass through explicit operand registers before reaching the PE. Products and sums wrap modulo 2^{16}.

IV. Mappings and Cost Model

A. Mappings

Let $L = \min(R,C)$. All mappings below are those of the library; each is loaded with one `CFG` per call (per column tile for matrix kernels).

Element-wise (diagonal). Vector a enters from the north and b from the west; PEs above the diagonal pass north values south, those below pass west values east, and the L diagonal PEs apply the operation. Because operands need up to $L-1$ hops, one chunk takes L steps and yields L results. Inputs stay fixed during the chunk, so the diagonal PEs recompute the same value until the last step. Partial chunks are padded with zeros, and only valid outputs are returned.

Dot product. $\mathrm{PE}(0,0)$ executes $r \leftarrow r + a_i b_i$ on its edge inputs; the host clears it once and supplies one pair per step. The partial sum stays local: local output-stationary accumulation is supported, and the other $RC-1$ PEs are idle.

Inclusive scan. $\mathrm{PE}(0,c)$ computes $\mathrm{op}(\mathrm{west}, \mathrm{north}_c)$ and passes the fold east; the previous block's last result enters west input 0 as a carry (the identity for the first block). One block of C elements takes C steps: $\mathrm{PE}(0,c)$ settles at step $c+1$ and recomputes thereafter. ADD, MUL, MIN and MAX folds are supported.

Matrix–vector ($y = Ax$). For each column tile, row 0 holds x_c in its immediates and multiplies the matrix row injected from the north; rows $1,\ldots,R-1$ pass products south (Fig. 5). After R steps the array holds $R\times C$ products, which the host reads, sums across columns and accumulates across column tiles. On a full tile, C PEs multiply and $(R-1)C$ act as delay cells; the host reduction is part of the algorithm. Column tiles whose x entries are all zero, and all-zero matrix tiles, are skipped. Convolution uses the same engine on implicitly generated Toeplitz tiles [25], visiting only tiles that intersect the band.

Forwarding limit. An interior PE cannot keep a sum s_t and forward two operands a_t east and b_t south in the same step: after a MAC its only visible register holds s_{t+1}. A one-MAC-per-PE-per-step output-stationary schedule therefore needs extra registers, routing PEs, time multiplexing or reconfiguration. This is a limit of that schedule on this PE, not an impossibility result for output-stationary mappings [18], [19].

B. Counting conventions

To avoid mixing units we count four quantities separately. A *PE slot* is one PE during one array step; a call with S steps offers RCS slots. A *useful PE operation* is one PE instruction whose result contributes to the answer: one addition per output for vector addition and scan, one MAC per element for the dot product, one multiplication per matrix entry for matrix–vector and convolution. *Scalar operations* count the multiplications

TABLE III
REQUEST AND REPLY SIZES IN BYTES; CHECKSUMS ARE INCLUDED.

Command	TX	RX	Total, 4×4
`ID`	1	5	6
`CFG`	$4P+2$	1	67
`WR`	$2E+2$	1	19
`RUN`	2	1	3
`RD`	1	$2P+1$	34
`RST`	1	1	2
`EXEC`	$2E+6$	$2t+2$	$24+2t$

TABLE IV
TRANSACTIONS PER LIBRARY-KERNEL CALL, EXCLUDING THE INITIAL IDENTIFICATION; NO RETRIES, NO SKIPPED TILES.

Kernel	v2	v3
Vector	$1+3K$	$1+K$
Dot	$3+2n$	$3+n$
Scan	$1+4Q$	$1+Q$
Dense matvec	$T_C[1+(2R+2)T_R]$	$T_C[1+RT_R]$

and additions of the scalar reference (a MAC is two), and *host operations* are additions performed by the host, such as the column sums of matrix–vector. Throughout, the useful fraction

$$\eta = \frac{\text{useful PE operations}}{RCS} \tag{1}$$

is distinct from the *static occupancy*, the fraction of PEs configured with an arithmetic opcode. For one 4×4 diagonal chunk, four of sixteen PEs are arithmetic (occupancy $1/4$) but four useful sums take four steps, so $\eta = 4/(16\cdot4) = 1/16$.

The abstract fabric ceiling is RCf_{clk}/D PE operations per second, 800×10^6 for 4×4, $D=2$ and 100 MHz. A mapping attains at most η times this ceiling, i.e. 50×10^6 useful operations per second for the diagonal, dot and scan mappings and 200×10^6 multiplications per second for matrix–vector, before any communication. The resident dot product, at $D+3=5$ clocks per element, attains at most 20×10^6 MACs per second.

C. Wire cost

Let $P=RC$, $E=R+C$ and t the number of registers selected by a tap mask. Table III gives request and reply sizes, including checksums. For a 4×4 diagonal chunk, `WR+RUN+RD` costs 56 bytes in three transactions, and `EXEC` with four taps 32 bytes in one. Table IV gives the resulting transaction counts of the library kernels for nonempty inputs, a cached identity (the initial identification, one transaction of six bytes, is excluded), no retries and no skipped tiles; $K=\lceil n/L\rceil$, $Q=\lceil n/C\rceil$, $T_R=\lceil M/R\rceil$ and $T_C=\lceil N/C\rceil$ for an $M\times N$ matrix. With B executed tile pairs in C_a active column tiles, matrix–vector and convolution need $C_a+(2R+2)B$ transactions in v2 and C_a+RB in v3. Every count reported in Section VI equals these closed forms; the analysis scripts check this for every geometry before plotting.

D. First-order time model

At 115 200 Bd with 8N1 framing a byte occupies $t_b = 10/115\,200\,\text{s} \approx 86.8\,\mu\text{s}$, i.e. 8 680 clocks at 100 MHz. For a call with B wire bytes, J transactions and S array steps we use

$$T \approx Bt_b + J\tau + S\frac{D}{f_{\text{clk}}} + T_{\text{host}}, \tag{2}$$

where τ is the turnaround added per transaction beyond serialization (USB bridge, driver and operating system; e.g. latency timers of USB serial bridges [26]). Serialization and turnaround are counted once each: τ excludes the bytes' own transmission time. The model makes three simplifying assumptions: τ is the same for every command, although buffering can make it depend on the reply size; controller cycles outside steps are neglected, which inspection of the controller bounds to a few clocks per word against 8 680 per byte; and T_{host}, unmeasured, is omitted from comparisons.

Under these assumptions, fused execution (v3) is faster than the unfused sequence (v2) if and only if

$$\tau > \tau^* = t_b\frac{B_{\text{v3}}-B_{\text{v2}}}{J_{\text{v2}}-J_{\text{v3}}}, \tag{3}$$

with $\tau^* < 0$, i.e. always, when fusion also reduces bytes. A 16 ms turnaround, used below as one scenario, is a common latency-timer setting, not a property measured on this platform.

V. METHODOLOGY

Counters. Byte, transaction and step counts are obtained by calling the public C API on the library's in-process device model, which implements the byte-level protocols of Section III without a clock. Each point is one call (or one sequence of calls) after the identification exchange, with configuration, resets, computation and read-back included; the driver aborts if any retransmission occurs, and none did. Every result is compared with an independent scalar reference and an output canary before it is recorded. Counts are deterministic for given inputs, so no confidence intervals apply; resident byte counts depend slightly on the data through byte stuffing. The analysis scripts recompute every transaction and step count from the closed forms of Section IV-C and stop on any mismatch. These are counts, not times: the model has no notion of elapsed time.

Model-to-RTL agreement. Counts transfer to the hardware only if the model and the RTL exchange the same bytes. GHDL [27] benches replay 504 model-generated legacy transactions byte-exactly against the controller and array for $D=1$–4, compare 1 024 PE results over all opcode and selector pairs, replay 886 framed transcript events for the block path and inject resets during upload and computation. Agreement of two models built by the same author cannot exclude a shared misunderstanding of the intent [28]; the scalar references address this for numerical results.

Formal verification. Safety properties written in PSL [29] are bound to the production VHDL, imported through the GHDL plugin for Yosys [30], and checked by SymbiYosys with Yices [31] (versions pinned by the repository). PROVED means that the bounded base case and temporal k-induction both

succeed [32], [33]; BOUNDED_PASS means a finite bound or a reached cover, which shows reachability only. Negative tasks and mutated design copies [34] must fail; this guards against vacuous properties.

Implementation. AMD Vivado 2026.1 implements the design on an Artix-7 `xc7a100tcsg324-1` (Nexys 4 DDR board, 100 MHz oscillator). A run passes only if synthesis messages, the presence of every expected constrained register and endpoint, synchronisers, clock-domain crossings, design rules, setup and hold all pass; each run archives its inputs, commands, reports and source hashes.

Table V states what each class establishes and on which source revision it was obtained. Results obtained on an earlier snapshot are reported as such.

VI. Results

A. RQ1: when does fusion pay?

Table VI lists the counted cost of the five kernels. Fusion divides the number of transactions by 1.99–3.95×. It also reduces bytes for vector addition and scan, whose unfused form reads back all sixteen registers per chunk, but increases bytes by 2.5–8.9% for dot product, matrix–vector and convolution: per element or per tile the fused transactions cost two to four bytes more than the sequences they replace, because step, flag, mask and status bytes outweigh the removed command bytes and these kernels have little read-back to omit. These ratios are count ratios, not speedups.

By Eq. (3), fusion is therefore never slower in the model for vector and scan, and faster for the other three kernels only when the turnaround exceeds $\tau^* = 173.6\,\mu$s (dot), $57.9\,\mu$s (matrix–vector) and $57.9\,\mu$s (convolution), i.e. two, two thirds and two thirds of a byte time. Fig. 6 shows the full dependence: for $\tau \to 0$ the ratio tends to the byte ratio, for large τ to the transaction ratio. Fig. 7 shows why the device term plays no role: on the 4×4 mesh at $D = 2$ and 100 MHz, SD/f_{clk} is at most 2.8×10^{-5} of the serialization time of the same call. Within the model, a higher clock or a smaller D changes total time by less than that fraction. Whether fusion pays on a real system is decided by the measured distribution of τ, which this study could not obtain.

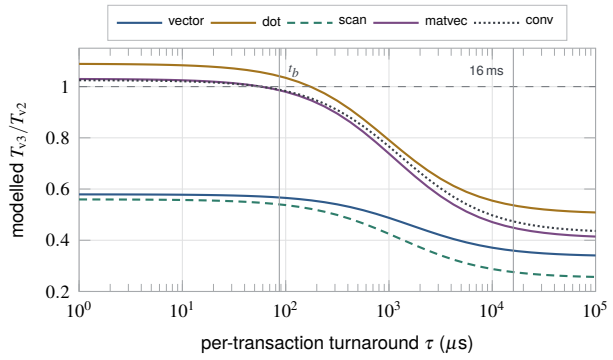


Fig. 6. Modelled call-time ratio of fused (v3) to unfused (v2) execution as a function of the turnaround τ, from the measured byte and transaction counts of Table VI and $T = Bt_b + J\tau$ at 115 200 Bd. Below the dashed line fusion is faster. Vertical lines mark one byte time t_b and a 16 ms USB latency timer. Model, not measurement.

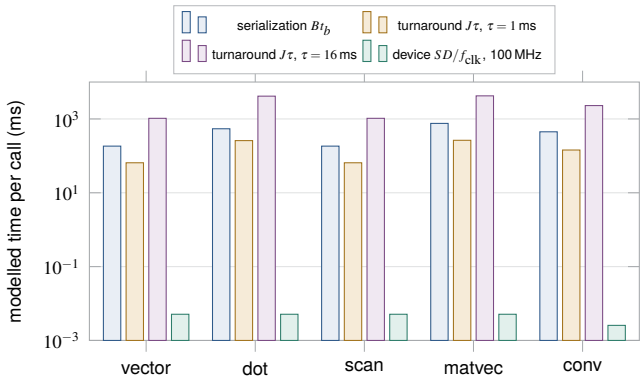


Fig. 7. Terms of the time model (2) for one call of each kernel (protocol v3, 4×4, $D = 2$), computed from the counted bytes B, transactions J and steps S. Two turnaround values are scenarios, not measurements. The device term is four to five orders of magnitude below serialization; host time is not modelled.

B. RQ2: geometry, mapping and useful work

Fig. 8 separates three quantities for the five 16-PE shapes, using the same problem sizes as Table VI and counted array steps. The useful fraction η is $1/16$ for vector addition, dot product and scan on every shape: the diagonal produces L results in L steps on RC PEs, the dot product uses one PE per step, and a scan block of C elements occupies the C PEs of row 0 for C steps. Matrix–vector reaches $\eta = 1/R$ (all sixteen PEs multiply on 1×16, one on 16×1), and convolution less because band tiles contain structural zeros. Static occupancy (panel b) is up to sixteen times larger than η for scan on 1×16, where all sixteen PEs are configured but only one result per step is new, and should not be used as a throughput proxy.

For vector, dot and scan the shape changes only communication. The same 256-element vector addition needs 2 115 bytes on 4×4 but 11 331 bytes on the strips, because every transfer carries $R + C$ edge words for at most $\min(R, C)$ results; scan and matrix–vector are cheapest on 1×16 (1 251 and 4 870 bytes), and matrix–vector and convolution are most expensive on 16×1 (47 200 and 60 608 bytes), where they also have the lowest η. The number of steps of matrix–vector ranges from 64 to 1 024 across shapes, but, as shown for RQ1, steps hardly matter on this link. No shape minimises traffic for every kernel; for a known dominant kernel the shape should follow its edge utilisation.

C. RQ3: residency, resources and verification effort

Communication. The fused legacy dot product sends one 24-byte exchange per element: 4 bytes of operands, 12 bytes of unused edge words (the payload always carries $R + C$ words) and 8 bytes of command, step, flag, mask, checksum and status. Uploading both vectors in 64-word frames before a single computation removes the padding and most per-element overhead. Fig. 9 separates this batching effect from reuse across calls. For eight dot products of length 256, the fused legacy kernel transfers 49 976 bytes in 2 072 transactions; re-

TABLE V

EVIDENCE CLASSES, THE SOURCE REVISION ON WHICH EACH WAS OBTAINED, AND WHAT IT DOES AND DOES NOT ESTABLISH. "CURRENT" MEANS THAT THE FILE HASHES RECORDED BY THE RUN EQUAL THOSE OF THE EVALUATED REVISION FOR EVERY FILE THE RUN USES.

Evidence	Run and revision	Establishes	Does not establish
Counters with scalar references	Recorded for this paper; current library	Exact bytes, transactions and steps for the stated inputs; closed forms	Elapsed time; behaviour on untested inputs
Model/RTL replay (GHDL)	Archived full simulation suite; RTL and benches current	Byte-exact agreement on recorded transcripts, $D = 1$–4	Equivalence; errors shared by model and RTL
Software suites (3 519 checks, sanitizers)	Archived; library current, command-line tool one revision earlier	Tested API, transport and fault-injection behaviour	Behaviour of a physical bridge; path coverage
Formal proofs and covers	Re-executed for this paper; current	Asserted safety on enumerated instances; reachability of covers	Unasserted properties; other parameters; unconditional liveness
Routed STA, legacy 4×4	Re-executed for this paper at 10 ns and 5.859 ns; current	Setup/hold closure of one netlist under its constraints	Correctness of the exception premise (Section VI-D); board timing
Routed STA, other runs	Archived; block build current; geometry sweep and period search on an earlier wrapper	As above, for those configurations	Statistical shape effects; a maximum frequency
Board measurement	Not run	–	–

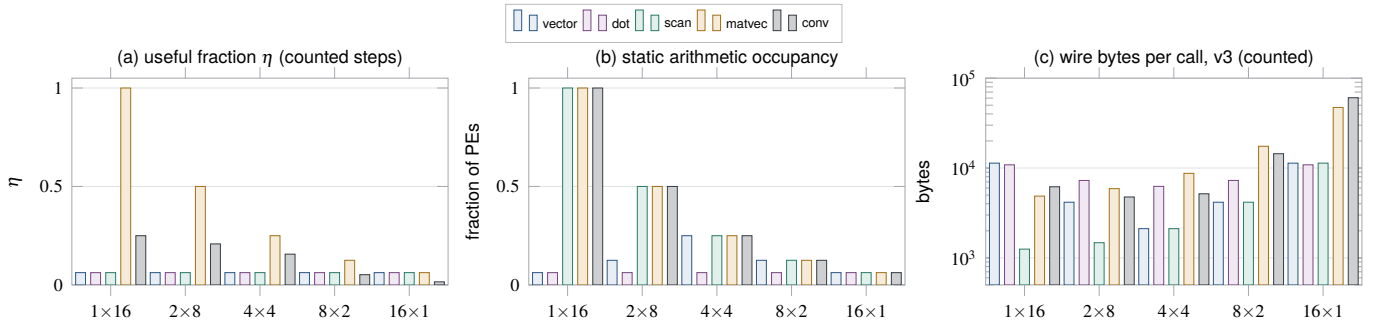


Fig. 8. Geometry and mapping for the five 16-PE shapes (workloads of Table VI). (a) Useful fraction η of PE slots, Eq. (1), from counted steps: vector, dot and scan stay at $1/16$ on every shape; matrix–vector reaches $1/R$. (b) Static fraction of PEs configured with an arithmetic opcode, which overstates efficiency wherever PEs recompute settled values or forward. (c) Counted wire bytes of one v3 call. For vector, dot and scan the shape changes only communication; for the matrix kernels it changes both.

TABLE VI

COUNTED WIRE COST OF ONE PUBLIC-API CALL, 4×4, PROTOCOL V2 VERSUS V3. VECTOR AND SCAN: ADD, $n = 256$; DOT: $n = 256$; MATRIX–VECTOR: DENSE NONZERO 32×32; CONVOLUTION: FULL OUTPUT, $k = 5$, $n = 64$. CONFIGURATION, RESETS, COMPUTATION AND READ-BACK ARE INCLUDED; THE INITIAL IDENTIFICATION IS EXCLUDED; NO RETRIES; NO TILES SKIPPED. BYTES ARE TX+RX.

	Transactions		Bytes	
Workload	v2	v3	v2	v3
Vector, 256	193	65	3 651	2 115
Dot, 256	515	259	5 735	6 247
Scan, 256	257	65	3 779	2 115
Matvec, 32×32	648	264	8 472	8 728
Conv, $k = 5, n = 64$	336	144	5 040	5 168

uploading both vectors for every call (no reuse) needs 10 874 bytes in 82 transactions, a 4.60× reduction attributable to batching and to sending only the needed operands; reusing one vector needs 6 368 bytes; reusing both needs 1 857 bytes in 26 transactions (26.91× fewer bytes, −96.28%, and 79.69× fewer transactions). The last case repeats an identical computation on identical data, whose result the host could have cached; it measures the amortisation of uploads, not a realistic workload. Resident byte counts depend on the data through byte stuffing (an archived run with different operands counted 1 856 instead of 1 857 bytes). Since the device term is negligible (RQ1), the time benefit predicted by Eq. (2) follows the byte and transaction reductions, but no speedup was measured.

Resources. Enabling the extension adds +68.6% LUTs and +320.5% flip-flops after synthesis (Table VII). The archived reports have no per-module breakdown; by construction the added state comprises a 144-byte receive buffer, the cached response, CRC and session registers and the operand registers, but their individual shares were not measured. The two logical 1024×16 banks map to two RAMB18 primitives counted by Vivado as one block-RAM tile. Only one PE computes during `DOT`, so residency trades array utilisation for traffic.

Verification effort. The extension added four property families with 163 assertion cells (sequencer 24, memories 31, receiver 22, integrated controller 86) and 15 positive proof tasks taking 175 s of solver time in the full suite, against 23 tasks and 237 s for the entire legacy design (Table VIII). Its integration also required two timing-driven redesigns before routed timing closed (Section VI-D). These figures are a coarse proxy of effort, not a complexity metric.

TABLE VII

RESOURCE COST OF THE RESIDENT-BLOCK EXTENSION, 4×4, $D = 2$, 10 NS, VIVADO 2026.1, `xc7a100tcsg324-1`. POST-SYNTHESIS ROWS USE IDENTICAL SOURCES, TOOL AND `FLATTEN_HIERARCHY NONE`; THE ROUTED ROW COMES FROM A SEPARATE RUN WITH THE DEFAULT STRATEGY. THE LEGACY POST-SYNTHESIS COUNT DIFFERS FROM TABLE IX (5 017 VERSUS 4 999 LUTS) BECAUSE THAT TABLE USES THE DEFAULT SYNTHESIS STRATEGY.

Build	LUT	FF	DSP	BRAM
Legacy, post-synthesis	5 017	1 034	16	0
With blocks, post-synthesis	8 459	4 348	16	2 RAMB18
Difference	+68.6%	+320.5%	0	+1 tile
With blocks, post-route	8 164	4 346	16	2 RAMB18

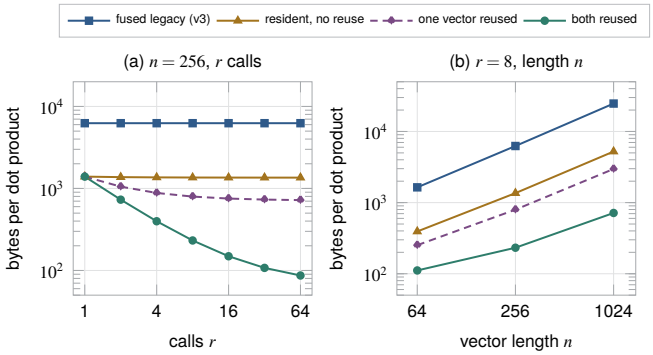


Fig. 9. Wire bytes per dot product through the public API (4×4 model; preflight, `HELLO`, uploads, computations and results included; every result checked against a scalar reference). "No reuse" uploads both vectors before every call and isolates batching; "both reused" repeats an identical computation, an amortisation microbenchmark whose result the host could also have cached. Counts, not times.

D. From proved capture guards to timing exceptions

A PE result is a combinational function of the PE's own register, its neighbours' registers, its configuration word and the edge registers. The controller captures it only every D clocks, so the design applies a D-cycle setup and $(D-1)$-cycle hold multicycle exception to these paths [35], [36]. Such an exception is correct only if the sources are stable for D cycles before every capture; a positive slack under the exception does not establish that premise. We document the argument as a chain of four checkable links.

(1) Functional premise, proved on the RTL. For the controller, properties proved by k-induction on every enumerated instance (Table VIII) state that captures are at least D clocks apart, that the first capture after a datapath reset is at least D clocks later, that configuration and edge registers change only in their receive states or on reset, and that at least $D-1$ clocks separate such a change from the next non-reset capture. The PE and array proofs establish that PE registers change only on step or reset. For the block extension, analogous guards on the operand and ownership registers are proved for $D = 1$–4. The reachable-state lemmas these inductions need are asserted and proved, never assumed: the controller tasks have no assumptions at all, and the integrated block controller assumes only an initial reset; it proves the range bound ($count \leq 1024$) that the standalone sequencer and memory tasks assume.

(2) Constraint scope. The constraint script names every expected source register bit—for 4×4, 256 PE result bits, 416 used configuration bits, 64 west and 64 north bits, i.e. 800 launch registers (833 with the block operand and ownership registers)—and the 256 PE data inputs as endpoints. Exceptions are applied from clock pins to data pins only, so clock-enable and reset pins keep single-cycle checks, as do RAM to operand-register paths. A run fails if any expected register is missing from the synthesized netlist or any unexpected one matches.

(3) Static timing. The routed netlist must meet setup and hold with these exceptions, and must pass checks on endpoint coverage, unconstrained paths, synchronisers and clock-domain crossings.

(4) Residual gap. The link between (1) and (2) is by register name: we did not check formally that synthesis preserved the RTL register boundaries, and the proofs cover enumerated geometries and D values, not all parameters. Metastability of the asynchronous inputs is outside both proofs and STA [37].

Table IX lists the implementation results. On the current revision the legacy 4×4 design closes at 10 ns with the same slack and resources as an archived run on an earlier snapshot, which differed only in a top-level wrapper that is pure wiring when the extension is disabled. A heuristic period search on that earlier snapshot found a passing candidate at 5.859 ns ($1000/T = 170.7$ MHz); implementing the current revision at that period again passes every gate with identical slack and resources. This is a constraint result for one placement, not a maximum frequency and not a change of the board's 100 MHz oscillator. With $D = 2$, the corresponding step rate is f_{clk}/D, and kernel throughput is lower still by η and, dominantly, by the link (RQ1). The block extension closed at 10 ns only after two retained failing designs (-3.580 and -2.867 ns), which led to explicit operand and ownership registers with proved guards. The geometry sweep uses one placement per shape, so differences in slack between shapes are not statistically interpretable.

VII. LIMITATIONS AND THREATS TO VALIDITY

No physical measurement. No FPGA board was available. Every time in this paper is a model value; τ, T_{host}, the behaviour of the USB–UART bridge, board-level reliability and power are unmeasured. A claim that fusion or residency is faster on hardware requires measured distributions of τ and call times on a programmed board, a stated warm-up and repetition policy, and a timed CPU baseline for the same kernels [38]. The supplement fixes such a procedure and the form of its report.

Model validity. Eq. (2) assumes one turnaround for every command; buffering may make τ depend on the command and reply size, which would shift the break-even values. Counts come from a clockless model; their transfer to the RTL rests on byte-exact replay of recorded transcripts, not on an equivalence proof.

Workloads. Five fixed kernels with fixed sizes and inputs are studied; matrix zero skipping and resident byte stuffing make counts input-dependent. The repeated identical dot product is

TABLE VIII

FORMAL RESULTS ON THE PRODUCTION VHDL, RE-EXECUTED FOR THIS PAPER ON THE CURRENT SOURCES (SYMBIYOSYS WITH YICES 2.7.0, GHDL 6.0.0 PLUGIN, YOSYS 0.67). PROVED: BOUNDED BASE CASE AND k-INDUCTION AT THE STATED DEPTH. INSTANCES ARE ENUMERATED; NO RESULT IS A THEOREM OVER ALL PARAMETERS. ASSERTION COUNTS ARE CELLS IN THE SOLVER MODEL.

Family	Instances and conclusion	Assumptions	Method	Assert.	Result
PE datapath	All 16 opcodes, both 8-way muxes, all 16-bit operands; reset priority and hold	Initial reset	k-ind., $k = 4, 8$	1, 4	PROVED
Array wiring	1×1 monolithic; 1×2, 2×1, 2×3, 3×2, 4×4, 1×16, 16×1 by composition	Initial reset; PE transitions as proved component guarantees	k-ind., $k = 4$	1–16	PROVED
Legacy controller	1×1 with $D = 1$–4; 2×3, 3×2, 4×4, 1×16, 16×1 with $D = 2$; capture count/spacing, guards, rejected `EXEC`, `EXEC` reply, watchdog, TX handshake, ranking functions	None	k-ind., $k = 16$	77–78	PROVED
UART	TX and RX digital models at divisors 3 and 868	None	k-ind., $k = 16$	21, 31	PROVED
Block sequencer	$D = 1$–4: count, cadence, operand/ownership guards, result capture	*count* ≤ 1024 (port subtype)	k-ind., $k = 16$	24	PROVED
Block memories	Two 1024×16 banks, arbitrary address, $D = 1$–4: retention and read order	Same	k-ind., $k = 16$	31	PROVED
Frame receiver	Timeout 3 and 10^8: escaping, length, CRC accept/reject, watchdog	None	k-ind., $k = 8$	22	PROVED
Block controller	Receiver and sequencer connected, $D = 1$–4 and $T = 10^8$: effects only from validated frames, cache, exact replay without duplicated effects, sequence exhaustion	Initial reset	k-ind., $k = 6$	86	PROVED
Negative controls	One false assertion and nine mutated copies (adder, step, `EXEC` byte order, north link, UART TX/RX, CRC bypass, RAM bank, early capture)	As above	BMC, 4–40	–	10/10 FAIL as required
Reachability	18 explicit cover tasks, including a complete block transcript driver	Witness-only stimuli	BMC, 40–200	–	18 BOUNDED_PASS

TABLE IX

ROUTED IMPLEMENTATION RESULTS, VIVADO 2026.1, `xc7a100tcsg324-1`, $D = 2$, DEFAULT STRATEGY, NO FLOORPLAN, ONE PLACEMENT EACH. ALL RUNS PASSED EVERY GATE (SETUP, HOLD, ENDPOINT AND CONSTRAINT COVERAGE, SYNCHRONISERS, CDC, DRC). LUT: POST-SYNTHESIS / POST-ROUTE. REV.: C = CURRENT REVISION; E = EARLIER SNAPSHOT WHOSE TOP-LEVEL WRAPPER DIFFERS ONLY BY WIRING; "THIS PAPER" MARKS RUNS EXECUTED FOR THIS REVISION.

Design	T/ns	WNS	WHS	LUT	FF	Rev.
4×4, this paper	10	+1.862	+0.098	4 999/4 971	1 034	C
4×4, this paper	5.859	+0.177	+0.122	4 999/4 975	1 034	C
4×4, period search	5.859	+0.177	+0.122	–/4 975	1 034	E
4×4 with blocks	10	+0.427	+0.060	8 262/8 164	4 346	C
1×16	10	+2.433	+0.081	5 017/4 999	1 179	E
2×8	10	+0.775	+0.105	5 003/4 974	1 066	E
4×4	10	+1.862	+0.098	4 999/4 971	1 034	E
8×2	10	+2.350	+0.130	5 002/4 975	1 066	E
16×1	10	+1.247	+0.098	5 010/4 993	1 179	E

an amortisation microbenchmark. The geometry comparison holds problem sizes fixed but not mappings optimised per shape.

Verification scope. The proofs cover selected safety properties on enumerated instances; completion of transactions is shown only under explicit delivery and service obligations. The legacy protocol's recovery from arbitrary byte insertion or loss is not proved, and the formal properties do not cover the C library. The link between proved RTL registers and constrained netlist cells is by name (Section VI-D).

Implementation evidence. Each geometry was placed and routed once, on an earlier snapshot whose wrapper differs from the current one only by wiring; only the 4×4 point was re-implemented on the current revision. The period search is heuristic and was not repeated; its candidate was re-checked on the current revision. Resources and frequencies are specific to one device, tool version and strategy, and are not comparable with other work.

Same-author references. The device model, RTL, test benches and properties were written by the same author; independent scalar references and mutation tests reduce, but do not remove, the risk of a shared misunderstanding.

VIII. ARTIFACT AVAILABILITY

The design, host library, device model, test benches, formal properties, implementation scripts and this paper's sources are kept in one repository, publicly hosted at https://github.com/4137314/ald-cgra. At the time of writing, the public branch predates the revision evaluated here, and the repository has no license; an immutable published revision and a license are prerequisites for reuse and are not yet in place. Within the repository, the paper build (`make -C doc -f doc.mk`) only reads archived data: it re-verifies provenance hashes and every closed-form count, and fails on any mismatch. Counts are regenerated deliberately with `make -C doc -f doc.mk measure-analysis`, which compiles an isolated copy of the library. Formal and implementation campaigns are separate

commands of the hardware flow; each run records its inputs, commands, tool versions, reports and source hashes, and the runs cited here are archived with the paper's review material.

IX. Conclusion

For a small step-controlled CGRA behind a serial link, communication, not the fabric, determines the cost of every kernel studied. Exact counts and a first-order model answer the three questions within that scope. Fusion reduces transactions by 1.99–3.95×; it is never slower for kernels whose read-back it shrinks, and for the others it pays only once the per-transaction turnaround exceeds between two thirds of a byte time and two byte times. Geometry changes traffic by up to an order of magnitude while useful work per PE slot stays at $1/16$ for three of five kernels, so static occupancy is a poor proxy for efficiency. Resident operands remove most of the traffic through batching alone (4.6×) and more through reuse, at a substantial cost in flip-flops and in verification effort.

Methodologically, the study shows that the premise of a multicycle timing exception can be proved on the production RTL and connected, by named registers, to the exceptions that STA checks; the remaining gap is the correspondence between RTL and netlist registers. The decisive missing evidence is physical: measured turnaround distributions, call times and a CPU baseline on a programmed board would turn the break-even analysis into a measured result.

References

[1] R. Hartenstein, "A decade of reconfigurable computing: A visionary retrospective," in *Design, Automation and Test in Europe (DATE)*, 2001, pp. 642–649. [Online]. Available: https://doi.org/10.1109/DATE.2001.915091

[2] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Computing Surveys*, vol. 52, no. 6, pp. 118:1–118:39, 2019. [Online]. Available: https://doi.org/10.1145/3357375

[3] A. Podobas, K. Sano, and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *IEEE Access*, vol. 8, pp. 146 719–146 743, 2020. [Online]. Available: https://doi.org/10.1109/ACCESS.2020.3012084

[4] S. A. Chin, K.-P. Niu, M. Walker, S. Yin, A. Mertens, J. Lee, and J. H. Anderson, "Architecture exploration of standard-cell and FPGA-overlay CGRAs using the open-source CGRA-ME framework," in *Proc. Int. Symp. Physical Design (ISPD)*, 2018, pp. 48–55. [Online]. Available: https://doi.org/10.1145/3177540.3177553

[5] C. Liu, H.-C. Ng, and H. K.-H. So, "QuickDough: A rapid FPGA loop accelerator design framework using soft CGRA overlay," in *Proc. Int. Conf. Field Programmable Technology (FPT)*, 2015. [Online]. Available: https://doi.org/10.1109/FPT.2015.7393130

[6] B. Mei, S. Vernalde, D. Verkest, H. De Man, and R. Lauwereins, "ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix," in *Field Programmable Logic and Application (FPL)*, ser. LNCS, vol. 2778. Springer, 2003, pp. 61–70. [Online]. Available: https://doi.org/10.1007/978-3-540-45234-8_7

[7] H. Singh, M.-H. Lee, G. Lu, F. J. Kurdahi, N. Bagherzadeh, and E. M. Chaves Filho, "MorphoSys: An integrated reconfigurable system for data-parallel and computation-intensive applications," *IEEE Transactions on Computers*, vol. 49, no. 5, pp. 465–481, 2000. [Online]. Available: https://doi.org/10.1109/12.859540

[8] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh, "HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect," in *54th Annual Design Automation Conference (DAC)*, 2017, pp. 45:1–45:6. [Online]. Available: https://doi.org/10.1145/3061639.3062262

[9] S. C. Goldstein, H. Schmit, M. Moe, M. Budiu, S. Cadambi, R. R. Taylor, and R. Laufer, "PipeRench: A coprocessor for streaming multimedia acceleration," in *26th International Symposium on Computer Architecture (ISCA)*, 1999, pp. 28–39. [Online]. Available: https://doi.org/10.1109/ISCA.1999.765937

[10] V. Govindaraju, C.-H. Ho, and K. Sankaralingam, "Dynamically specialized datapaths for energy efficient computing," in *17th International Symposium on High Performance Computer Architecture (HPCA)*, 2011, pp. 503–514. [Online]. Available: https://doi.org/10.1109/HPCA.2011.5749755

[11] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A reconfigurable architecture for parallel patterns," in *44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 389–402. [Online]. Available: https://doi.org/10.1145/3079856.3080256

[12] S. A. Chin, N. Sakamoto, A. Rui, J. Zhao, J. H. Kim, Y. Hara-Azumi, and J. Anderson, "CGRA-ME: A unified framework for CGRA modelling and exploration," in *28th International Conference on Application-Specific Systems, Architectures and Processors (ASAP)*, 2017, pp. 184–189. [Online]. Available: https://doi.org/10.1109/ASAP.2017.7995277

[13] Embedded Systems Laboratory, EPFL, "OpenEdgeCGRA: An open-hardware coarse grained reconfigurable array for accelerated computation on the edge," Source repository, accessed 2026-10-15. [Online]. Available: https://github.com/esl-epfl/OpenEdgeCGRA

[14] M. H. Aspros, J. Sapriza, G. Ansaloni, and D. Atienza, "A flexible framework for early power and timing comparison of time-multiplexed CGRA kernel executions," arXiv:2504.01672v3, 2025. [Online]. Available: https://arxiv.org/abs/2504.01672v3

[15] H. T. Kung and C. E. Leiserson, "Systolic arrays (for VLSI)," in *Sparse Matrix Proceedings 1978*, I. S. Duff and G. W. Stewart, Eds. SIAM, 1979, pp. 256–282.

[16] H. T. Kung, "Why systolic architectures?" *Computer*, vol. 15, no. 1, pp. 37–46, 1982. [Online]. Available: https://doi.org/10.1109/MC.1982.1653825

[17] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1–12. [Online]. Available: https://doi.org/10.1145/3079856.3080246

[18] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017. [Online]. Available: https://doi.org/10.1109/JPROC.2017.2761740

[19] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017. [Online]. Available: https://doi.org/10.1109/JSSC.2016.2616357

[20] W. Simpson, "PPP in HDLC-like framing," RFC 1662, Internet Engineering Task Force, Jul. 1994. [Online]. Available: https://doi.org/10.17487/RFC1662

[21] W. W. Peterson and D. T. Brown, "Cyclic codes for error detection," *Proceedings of the IRE*, vol. 49, no. 1, pp. 228–235, 1961. [Online]. Available: https://doi.org/10.1109/JRPROC.1961.287814

[22] P. Koopman and T. Chakravarty, "Cyclic redundancy code (CRC) polynomial selection for embedded networks," in *Proc. Int. Conf. Dependable Systems and Networks (DSN)*, 2004, pp. 145–154.

[23] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, 1984. [Online]. Available: https://doi.org/10.1145/357401.357402

[24] A. D. Birrell and B. J. Nelson, "Implementing remote procedure calls," *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, 1984. [Online]. Available: https://doi.org/10.1145/2080.357392

[25] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013. [Online]. Available: https://www.press.jhu.edu/books/title/10678/matrix-computations

[26] *Data Throughput, Latency & Handshaking*, Future Technology Devices International, Mar. 2024, AN232B-04 (v1.2). [Online]. Available: https://ftdichip.com/wp-content/uploads/2024/03/AN232B_04_DataThroughput_LatencyandHandshaking.pdf

[27] T. Gingold, "GHDL: A VHDL 2008/93/87 simulator," accessed 2026-09-22. [Online]. Available: https://github.com/ghdl/ghdl

[28] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[29] *IEEE Standard for Property Specification Language (PSL)*, IEEE, 2010, iEEE Std 1850-2010.

[30] C. Wolf, "Yosys open SYnthesis suite," accessed 2026-10-15. [Online]. Available: https://yosyshq.net/yosys/

[31] B. Dutertre, "Yices 2.2," in *Computer Aided Verification (CAV)*, ser. LNCS, vol. 8559. Springer, 2014, pp. 737–744. [Online]. Available: https://doi.org/10.1007/978-3-319-08867-9_49

[32] A. Biere, A. Cimatti, E. M. Clarke, and Y. Zhu, "Symbolic model checking without BDDs," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. LNCS, vol. 1579. Springer, 1999, pp. 193–207. [Online]. Available: https://doi.org/10.1007/3-540-49059-0_14

[33] M. Sheeran, S. Singh, and G. Stålmarck, "Checking safety properties using induction and a SAT-solver," in *Formal Methods in Computer-Aided Design (FMCAD)*, ser. LNCS, vol. 1954. Springer, 2000, pp. 108–125. [Online]. Available: https://doi.org/10.1007/3-540-40922-X_8

[34] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978. [Online]. Available: https://doi.org/10.1109/C-M.1978.218136

[35] *Vivado Design Suite User Guide: Using Constraints*, AMD/Xilinx, 2022, UG903 (v2022.2). [Online]. Available: https://docs.amd.com/r/2022.2-English/ug903-vivado-using-constraints

[36] J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs: A Practical Approach*. Springer, 2009. [Online]. Available: https://doi.org/10.1007/978-0-387-93820-2

[37] R. Ginosar, "Metastability and synchronizers: A tutorial," *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, 2011. [Online]. Available: https://doi.org/10.1109/MDT.2011.113

[38] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2015, pp. 73:1–73:12. [Online]. Available: https://doi.org/10.1145/2807591.2807644